Plan and Execute

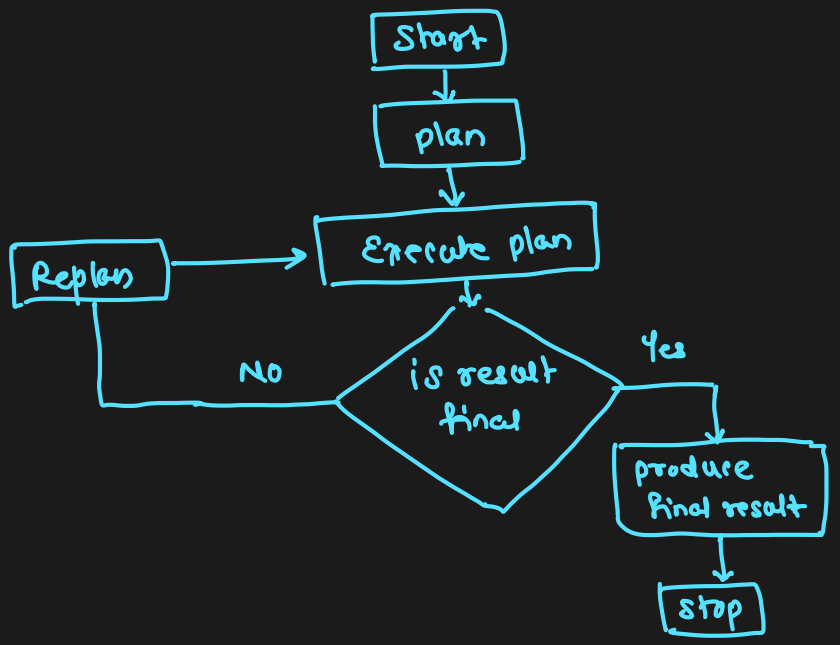
Introduction



- The Plan-and-Execute pattern separates "decide what to do" from "do it."
- An agent first produces a plan—a structured description of the work—then executes against that plan
- The plan becomes a durable artifact that is reviewable, modifiable, checkpointable, and resumable
- This differs fundamentally from reactive patterns where the agent jumps from request to execution without explicit planning
- For complex tasks like multi-file refactors, architectural changes, or multi-step workflows, skipping the planning phase produces edits that don't account for dependencies, missing steps discovered only after partial execution, and no recovery point if execution fails partway through

Architecture

- **Planner**
  - An LLM that parses the user request, understands intent, and generates an executable task plan—often organized as a Directed Acyclic Graph (DAG) representing task sequence and dependencies
- **Executor**
  - Executes tasks from the plan
  - May be a simple function caller or a full ReAct agent with tool access
- **Replanner**
  - Evaluates updated state after execution.
  - If results are insufficient, replans additional tasks and resubmits them
  - If sufficient, prepares final results
- The workflow follows a loop
  - User Request → Planner → Task DAG → Executor → State Update → Replanner → (loop or respond)



Implementation Approaches

- **Basic Loop Implementation**
  - A straightforward LangGraph implementation uses plan and execute nodes in a cycle
  - The state typically tracks the task, current plan (remaining steps), and past steps with their results
  - The executor node takes the first step from the plan, executes it, and returns the updated state with that step removed and its result appended to history → state
- **Multi-Agent Implementation**
  - An alternative approach uses separate plan and execute agents without cycles
  - The plan agent's output becomes input for the execute agent, providing clear guidance through multi-step reasoning while achieving faster execution time compared to cyclic approaches
  - This is implemented by passing the plan as context in the execute agent's prompt
- **Model Tiering Strategy**
  - Production implementations often use different models for different roles
  - **Planner:** High-performance model for quality thinking and strategic decomposition
  - **Executor:** Cheaper, faster model for straightforward step execution
  - **Replanner:** Balanced model for judgment accuracy

Advanced Variants

- **LLMCompiler**
  - This architecture speeds up agent tasks by executing them in parallel through a DAG
  - It saves redundant token use by reducing LLM calls
  - The system has three components: a Function Calling Planner that formulates execution plans, a Task Fetching Unit that dispatches function calling tasks, and an Executor that runs tasks in parallel
  - Benchmarks show consistent latency speedups of up to 3.7x with approximately 9% cost savings
- **Plan & Solve**
  - A simpler variant where planning and execution are distinct phases
  - The agent produces a plan, then executes it without the replanning loop.
  - Suitable for tasks requiring structured decomposition but not dynamic adaptation
- **Hierarchical Planning (Hybrid)**
  - The most sophisticated systems combine Plan-and-Execute with ReAct
  - A high-level planner creates a strategic plan of complex sub-goals
  - Each sub-goal is then handled by a specialized ReAct agent for tactical adaptability
  - This leverages strategic foresight from Plan-and-Execute and tactical flexibility from ReAct

When to Choose Plan-and-Execute



- **Choose this pattern when**
  - The task is well-structured and the environment is stable
  - Cost and latency are major constraints—minimizing expensive LLM calls benefits batch processing
  - The sequence of operations is complex but deterministic
  - Long chains of dependent actions have predictable outcomes
- **Avoid when**
  - The task is exploratory and requires dynamic adaptation
  - The environment changes significantly after the plan is created (basic implementations will fail)
  - Each step's outcome fundamentally changes what should happen next

Comparison with ReAct



- The two patterns answer one question differently Does the agent need to know the full plan before it starts?"
- ReAct (Reasoning + Acting) is an adaptive pattern
  - The agent operates in a tight loop: think → take one action → observe the result → decide the next step
  - Every action is informed by what just happened
  - As Oracle's integration team puts it: "I don't know exactly where this leads — I'll figure it out as I go."
- Plan-and-Execute is a methodical pattern
  - A Planner agent decomposes the goal into an ordered list of steps, then an Executor agent runs those steps.
  - The Planner never calls tools—it only thinks. The Executor never improvises—it only acts on what the Planner prescribed
  - As Oracle explains: "I'll figure out the full plan first, then hand it off to be executed exactly as written."

Use ReAct When

- **Path is unpredictable**
  - The sequence of steps is unknown until the previous result is seen
  - e.g., "find all blocked orders and resolve each one"
- **Step count is variable**
  - Tasks requiring 1-5 tool calls where each call informs the next
- **Error recovery needs creativity**
  - The agent must handle unexpected tool failures creatively—try alternative lookups, reframe queries
- **Low-stakes tasks**
  - Acceptable for conversational or low-stakes tasks where the execution trace is less critical
- **Latency matters**
  - Lower latency for simple tasks—no upfront Planner call overhead

Use Plan and Execute when



- **Steps are knowable upfront**
  - The steps required are knowable from the goal alone—
  - e.g., "generate a monthly report by pulling data from three systems"
- **Full tool sequence is predictable**
  - Tasks with a known set of sequential steps where the sequence is predictable from the goal description
- **Failures follow predictable logic**
  - Failures follow predictable retry or fallback logic that can be encoded in the plan step definition
- **Auditability is required**
  - Required for regulated, high-stakes, or compliance-sensitive workflows where every step must be logged and inspectable
- **Determinism matters**
  - The same task must execute the same way every time—deterministic, repeatable, and comparable across runs